Ejercicios 1: procedimientos
Unidad 2: Procedimientos y clausuras

Eduardo Acuña Yeomans

Lenguajes de Programación
Semestre 2026-2

Lic. en Ciencias de la Computación
Universidad de Sonora

Esta hoja no se entrega y no se califica. Conviene hacerla completa: la tarea de esta unidad da por sabido lo que está aquí. Son cinco bloques, y las soluciones están al final.

Bloque 1	programar en PROC, sin tocar el intérprete
Bloque 2	predecir valores y encontrar variables libres, a mano
Bloque 3	agregar `letproc`: tu primer constructo con lexer, parser y pruebas
Bloque 4	sacar `let` del núcleo: se vuelve azúcar sobre `proc`
Bloque 5	recortar lo que guarda la clausura: cambia la representación, no el lenguaje

Instrucción: copia la carpeta `proc-02-procedimientos` con el nombre `proc-ejercicios`, y en su `Makefile` cambia la línea `PKG` a `PKG := proc-ejercicios`. Los bloques 3, 4 y 5 se hacen en esa copia, uno después del otro. Antes de empezar, `make test` tiene que decir `356 tests passed`.

Lo que escribas a mano (predicciones, variables libres, respuestas a preguntas) va en papel o en un archivo de notas, para compararlo con las soluciones del final. El código va en tu copia. `make test` y `make run` trabajan sobre la copia sin instalar nada. Si quieres usar `#lang proc` con tu copia en DrRacket, quita la versión instalada con `make uninstall` en su carpeta y corre `make install` en la copia.

Recuerda qué: lo que necesitas de la hoja de referencia

La especificación completa de PROC está en la hoja de referencia de `proc-02-procedimientos`. De ella, las dos reglas de los procedimientos y la del `let`:

$$\frac{}{\texttt{(value-of (proc-exp } x\ c\texttt{) } \rho\texttt{)} = \mathsf{clausura}(x, c, \rho)}\ [\text{proc}]$$

$$\frac{\texttt{(value-of } e_1\ \rho\texttt{)} = \mathsf{clausura}(x, c, \rho') \qquad \texttt{(value-of } e_2\ \rho\texttt{)} = v_2}{\texttt{(value-of (call-exp } e_1\ e_2\texttt{) } \rho\texttt{)} = \texttt{(value-of } c\ [x{=}v_2]\rho'\texttt{)}}\ [\text{call}]$$

$$\frac{\texttt{(value-of } e_1\ \rho\texttt{)} = v_1}{\texttt{(value-of (let-exp } x\ e_1\ c\texttt{) } \rho\texttt{)} = \texttt{(value-of } c\ [x{=}v_1]\rho\texttt{)}}\ [\text{let}]$$

El cuerpo de un procedimiento se evalúa en ρ', el entorno en que se creó, y no en ρ, el de la llamada. Es el **alcance léxico**.

Un constructo de azúcar se especifica con una regla de transformación, con la flecha $\rightsquigarrow$,

y su cláusula va en `desugar.rkt`. Desde esta unidad el lexer, el parser y las pruebas también los extiendes tú.

Bloque 1: programar en PROC

Estos programas se escriben en PROC tal como está. Pruébalos en el REPL, con `make run`, o en un archivo con `#lang proc`. Un procedimiento de PROC recibe un solo argumento: para dos, se escribe un procedimiento que devuelve otro, como `menos`.

```
let menos = proc (n) proc (x) -(x, n)
in ((menos 5) 12)
```

vale 7. A esta forma de recibir varios argumentos, uno a la vez, se le llama **currificación**.

1.1 Suma

Instrucción: escribe `suma`, que recibe dos números, uno a la vez, y devuelve su suma. Escríbela solo con la resta: sin `add`, sin `+` y sin `minus`.

```
((suma 3) 4)    =  7
((suma -3) -4)  =  -7
```

1.2 Componer

Instrucción: escribe `compone`, que recibe un procedimiento `f` y después un procedimiento `g`, y devuelve el procedimiento que, al recibir `x`, calcula `(f (g x))`: primero `g` y luego `f`. Con `doble` = `proc (n) *(n, 2)` y `resta1` = `proc (n) -(n, 1)`:

```
(((compone doble) resta1) 5)  =  8
(((compone resta1) doble) 5)  =  9
```

1.3 Invertir los argumentos

Instrucción: escribe `invierte-args`, que recibe un procedimiento currificado de dos argumentos y devuelve otro que hace lo mismo, pero recibe los argumentos al revés. Para cualquier `h`, `a` y `b`:

```
(((invierte-args h) a) b)  =  ((h b) a)
```

Con `resta` = `proc (x) proc (y) -(x, y)`:

```
(((invierte-args resta) 10) 3)  =  -7
```

Después, usa `invierte-args` y `resta` para escribir `quita3`, el procedimiento que le resta 3 a lo que recibe, sin escribir otro `proc`.

Bloque 2: predecir

2.1 El valor de un programa

Instrucción: para cada programa, predice su valor sin correrlo. Después córrelo y compara.

a.
```
let x = 1
in let f = proc (y) -(y, x)
   in let x = 10
      in (f x)
```

b.
```
let menos = proc (n) proc (x) -(x, n)
in let quita1 = (menos 1)
   in let quita10 = (menos 10)
      in -((quita1 100), (quita10 50))
```

c.
```
let dos-veces = proc (f) proc (x) (f (f x))
in let resta1 = proc (n) -(n, 1)
   in (((dos-veces dos-veces) resta1) 10)
```

d.
```
let x = 3
in let f = proc (x) proc (y) -(x, y)
   in ((f 10) x)
```

2.2 Variables libres y vinculadas

Una variable puede aparecer en una expresión de dos maneras. Está **vinculada** si algo adentro de la expresión le da su valor. Tres constructos lo hacen: `proc (x)` vincula a `x` en su cuerpo, `let x =` e_1 vincula a `x` en su cuerpo (no en e_1), y `unpack` vincula sus nombres en su cuerpo (no en la lista que desempaca). Si nada adentro de la expresión le da valor, la variable está **libre**, y su valor lo pone el entorno en que se evalúa la expresión. Las **variables libres** de una expresión son las que aparecen libres al menos una vez. Dos ejemplos, con las apariciones vinculadas en **azul** y las libres en **rojo**:

`proc (x) -(x, y)`	el `proc` vincula a `x`. A `y` nadie: es libre.
`let x = x in -(x, 1)`	el `let` vincula a `x` solo en su cuerpo. La de la derecha del `=` es libre.

Instrucción: escribe las variables libres de cada expresión. Fíjate en qué partes de cada constructo cubre su vinculación y en cuáles no.

a. `-(x, y)`
b. `proc (x) -(x, y)`
c. `let y = 3 in proc (x) -(x, y)`
d. `let x = x in -(x, 1)`
e. `proc (f) (f (g x))`
f. `let z = 5 in proc (x) let y = -(x, z) in -(y, w)`
g. `(proc (x) x x)`
h. `unpack a b = list(a, c) in -(a, b)`

Instrucción: en el programa de `f` y `g` que vale `-100`, ¿cuál es la variable libre del cuerpo `-(z, x)`? Cuando se llama `(f 1)`, ¿en qué entorno se busca su valor, según [call], y cuánto vale ahí?

```
let x = 200
in let f = proc (z) -(z, x)
   in let x = 100
      in let g = proc (z) -(z, x)
         in -((f 1), (g 1))
```

Bloque 3: `letproc`

Un `letproc` crea un procedimiento y le da nombre a la vez. Es azúcar.

`Expression ::= letproc Identifier (Identifier) = Expression in Expression`
`(letproc-exp` f x e_1 e_2`)`

letproc f **(**x**) =** e_1 **in** e_2 $\rightsquigarrow$ **let** f **= proc (**x**)** e_1 **in** e_2 [letproc]

Instrucción: en tu copia, escribe

1. En `lexer.rkt`, la palabra reservada **`letproc`**: el token, su regla, su lugar en `token-names` y su texto en `empty-token-texts`.
2. En `parser.rkt`, la producción.
3. En `ast.rkt`, el `struct letproc-exp`, con campos `f x e1 e2`, en sus tres lugares: el `contract-out`, la definición y `expression?`.
4. En `desugar.rkt`, la cláusula, que traduzca según [letproc].
5. Las pruebas. En `lexer-tests.rkt`, que **`letproc`** es un token y `letprocs` un identificador. En `parser-tests.rkt`, el árbol de un `letproc`. En `desugar-tests.rkt`, su traducción, comparada contra el `parse` del `let` que le corresponde. En `main-tests.rkt`, su valor.

```
letproc f (x) = -(x, 11) in (f (f 77))  =  55
```

Así lo escribe el REPL. La prueba de `main-tests.rkt` compara el resultado de `run` con `(intval 55)`, como las que ya trae.

Instrucción: predice con [letproc] qué pasa con **`letproc f (x) = (f x) in (f 1)`**, y después córrelo. ¿Puede el cuerpo de un `letproc` llamar al procedimiento que se está definiendo?

Bloque 4: `let` sale del núcleo

Un `let` se puede escribir con un procedimiento y una llamada: el cuerpo pasa a ser el cuerpo de un **`proc`**, y el valor del nombre, el operando.

let x **=** e_1 **in** e_2 $\rightsquigarrow$ **(proc (**x**)** e_2 e_1**)** [let]

Instrucción: en tu copia,

1. En `desugar.rkt`, cambia la cláusula de `let-exp`: ya no reconstruye el `let`, lo traduce según [let]. Muévela a la parte del azúcar.
2. En `interp.rkt`, borra la cláusula de `let-exp`. `value-of` ya no la necesita: después de `desugar` no queda ningún `let`.
3. Corre `make test`.

Dejan de pasar once de las pruebas que traía `proc-02`: nueve de `interp-tests.rkt` y dos de `desugar-tests.rkt`. También las tuyas de `letproc` que comparaban contra un árbol con `let`.

Instrucción: di por qué falla cada grupo y arréglalas. Las de `interp-tests.rkt` llaman a `value-of` con un `let-exp`: pásalas por `desugar` antes, que es lo que hace `run`. Las de `desugar-tests.rkt` esperan un árbol con `let`: escribe el árbol que sale ahora. Agrega una prueba de [let] en `desugar-tests.rkt`.

Instrucción: ¿en cuántos pasos de `desugar` se traduce `letproc f (x) = x in (f 1)`, y qué árbol queda al final? ¿Tuviste que cambiar la cláusula de `letproc` para que funcione?

Bloque 5: la clausura guarda solo lo que usa

Una clausura guarda el entorno de creación entero, aunque el cuerpo use una o dos de sus variables. En este bloque vas a recortarlo: la clausura va a guardar solo las variables libres del **`proc`**. Son las únicas que el cuerpo puede llegar a buscar en ese entorno. El parámetro lo pone la llamada, y lo que vincula un **`let`** o un **`unpack`** adentro del cuerpo lo pone el cuerpo mismo. La especificación no cambia. Cambia la representación, y ningún programa tiene que dar otro valor.

Recuerda qué: las variables libres

Son las del bloque 2. En el núcleo, después de **`desugar`** y con el bloque 4 hecho, los constructos que vinculan son dos: **`proc`** vincula su parámetro en su cuerpo, y **`unpack`** vincula sus nombres en su cuerpo, pero no en la lista que desempaca.

Instrucción: en tu copia, escribe

1. En la interfaz del entorno, `(has-binding? ρ x)`, que dice si ρ vincula `x`: su implementación en `env-impl-struct.rkt`, su contrato en `env.rkt` y sus pruebas en `env-tests.rkt`.
2. En `interp.rkt`, abajo de `value-of`, `free-vars`, que recibe una expresión del núcleo y devuelve la lista de sus variables libres, sin repetir. Una cláusula por constructo del núcleo.
3. También en `interp.rkt`, `restrict-env`, que recibe un entorno y una lista de nombres, y arma un entorno nuevo con solo los nombres de la lista que el entorno vincula, con sus mismos valores.
4. La cláusula de `proc-exp`, que guarde en la clausura ese entorno recortado a las variables libres del **`proc`**.
5. Corre `make test`.

Deja de pasar una sola prueba de las que traía `proc-02`.

Instrucción: di cuál es, por qué falla aunque ningún programa cambió de valor, y reescríbela para que compruebe lo que este bloque promete: que la clausura guarda las variables libres del cuerpo y no guarda las demás.

Instrucción: `let f = proc (x) y in 5` vale `5` en `proc-02`, aunque `y` no esté vinculada. ¿Qué pasaría con él si `restrict-env` buscara cada variable libre con `apply-env`, sin preguntar antes con `has-binding?`? Escribe la prueba que lo cuida.

Sugerencia: para revisar el bloque 2

Las respuestas de las variables libres del bloque 2 son lo que tu `free-vars` tiene que dar. Pasa cada expresión por `parse` y `desugar` antes, porque `free-vars` solo conoce el núcleo.

Soluciones

Clave de Respuesta: bloque 1

```
let suma = proc (x) proc (y) -(x, -(0, y))
let compone = proc (f) proc (g) proc (x) (f (g x))
let invierte-args = proc (f) proc (a) proc (b) ((f b) a)
```

Cada uno va seguido de **in** y del programa que lo usa. En `compone`, el orden importa: `((compone doble) resta1)` resta primero y duplica después, $(5-1)\times 2 = 8$. `quita3` es `((invierte-args resta) 3)`: el procedimiento que espera `x` y calcula `-(x, 3)`, así que `(quita3 10)` vale 7.

Clave de Respuesta: bloque 2, el valor

a. 9. La clausura de `f` guarda $[\texttt{x} = 1]\varnothing$. El operando `x` se evalúa en el entorno de la llamada, donde vale 10, y el cuerpo en el de creación, donde vale 1: $10 - 1$.

b. 59. `quita1` guarda `n` $= 1$ y `quita10` guarda `n` $= 10$. Las dos salen del mismo **proc**: $(100 - 1) - (50 - 10)$.

c. 6. `(dos-veces dos-veces)` es la clausura de **proc** `(x) (f (f x))` con `f` vinculada a `dos-veces`. Llamada con `resta1`, aplica `dos-veces` dos veces sobre `resta1`: el resultado resta 1 cuatro veces.

d. 7. El `x` del parámetro tapa al de afuera adentro de `f`, y la clausura de **proc** `(y)` guarda `x` $= 10$. El operando `x` de la llamada de afuera vale 3: $10 - 3$.

Clave de Respuesta: bloque 2, las variables libres

a.	`x`, `y`	e.	`g`, `x`
b.	`y`	f.	`w`
c.	ninguna	g.	`x`, la del operando
d.	`x`, la de e_1	h.	`a`, la de la lista, y `c`

En d, h y g la misma variable aparece vinculada y libre: **let** y **unpack** vinculan solo en su cuerpo, y en g el operando está fuera del **proc**. En el programa de `f` y `g`, la variable libre de `-(z, x)` es `x`. Por [call], el cuerpo se evalúa en el entorno guardado en la clausura de `f`, donde `x` vale 200.

Clave de Respuesta: bloque 3

```
;; lexer.rkt, la regla, antes de identifier
["letproc"
 (token-LETPROC)]
;; parser.rkt
[(LETPROC ID LPAREN ID RPAREN EQ expression IN expression)
 (letproc-exp $2 $4 $7 $9)]
;; ast.rkt
```

```
(struct letproc-exp (f x e1 e2) #:transparent)
;; desugar.rkt, en la parte del azúcar
[(letproc-exp f x e1 e2)
 (desugar (let-exp f (proc-exp x e1) e2))]
```

El token también va en **define-empty-tokens** y en **token-names**, y su texto en **empty-token-texts**. El contrato del **struct** es **symbol?** para **f** y **x**, y **expression?** para **e1** y **e2**.

letproc f (x) = (f x) in (f 1) es error: **la variable f no está vinculada**. Se traduce a un **let**, y el **proc** se evalúa en el entorno de afuera del **let**, donde **f** todavía no existe. Su clausura no lo guarda, así que el cuerpo no puede llamarse a sí mismo. **letproc** no es recursivo.

Clave de Respuesta: bloque 4

```
;; desugar.rkt, en la parte del azúcar
[(let-exp x e1 e2)
 (desugar (call-exp (proc-exp x e2) e1))]
```

Las nueve de **interp-tests.rkt** fallan con un error de **match**: **value-of** ya no tiene cláusula para **let-exp**. Se arreglan con **(value-of (desugar ...) ρ)**, importando **"../desugar.rkt"**. Las dos de **desugar-tests.rkt** son la del árbol que tenía que salir intacto, que empezaba con un **let** y ya no es del núcleo, y la de **add** adentro de un **let**, que ahora sale como llamada:

```
(check-equal? (desugar (parse "let x = 1 in -(x, 2)"))
              (parse "(proc (x) -(x, 2) 1)")
              "[let] se traduce a la llamada de un proc")
```

letproc f (x) = x in (f 1) se traduce en dos pasos: primero a **let f = proc (x) x in (f 1)** y después a **(proc (f) (f 1) proc (x) x)**. La cláusula de **letproc** no cambia: produce un **let-exp** y lo vuelve a pasar por **desugar**, que ahora lo traduce. Su prueba de árbol sí cambia, porque ya no sale un **let**.

Clave de Respuesta: bloque 5

```
;; env-impl-struct.rkt
(define (has-binding? env x)
  (match env
    [(empty-env)
     #f]
    [(extend-env y v env*)
     (or (equal? y x)
         (has-binding? env* x))]))

;; interp.rkt, la cláusula
[(proc-exp x e1)
 (define ρ* (restrict-env ρ (free-vars e)))
```

```
 (procval (closure x e1 ρ*))]

;; interp.rkt, abajo de value-of
(define (free-vars e)
  (match e
    [(or (const-exp _) (emptylist-exp))
     '()]
    [(var-exp x)
     (list x)]
    [(or (zero?-exp e1) (minus-exp e1) (car-exp e1) (cdr-exp e1)
         (null?-exp e1) (not-exp e1))
     (free-vars e1)]
    [(or (diff-exp e1 e2) (mul-exp e1 e2) (quotient-exp e1 e2)
         (equal?-exp e1 e2) (less?-exp e1 e2) (cons-exp e1 e2)
         (or-exp e1 e2) (call-exp e1 e2))
     (union (free-vars e1) (free-vars e2))]
    [(if-exp e1 e2 e3)
     (union (free-vars e1) (union (free-vars e2) (free-vars e3)))]
    [(unpack-exp xs e1 e2)
     (union (free-vars e1) (remove* xs (free-vars e2)))]
    [(proc-exp x e1)
     (remove x (free-vars e1))]))

(define (union xs ys)
  (remove-duplicates (append xs ys)))

(define (restrict-env ρ xs)
  (cond
    [(null? xs)
     (empty-env)]
    [(has-binding? ρ (first xs))
     (extend-env (first xs) (apply-env ρ (first xs))
                 (restrict-env ρ (rest xs)))]
    [else
     (restrict-env ρ (rest xs))]))
```

Una cláusula de `match` con `or` atiende varios constructos de la misma forma. Escribir una cláusula por constructo también está bien. En el contrato de `env.rkt`: `[has-binding? (-> env? symbol? boolean?)]`.

La prueba que falla es la de [proc] en `interp-tests.rkt`, que compara el valor con `(procval (closure 'y (var-exp 'y) ρx))`. Ahora la clausura guarda un entorno nuevo, el recortado, y no `ρx`. Los entornos no son `#:transparent`, así que `equal?` los compara por identidad: son distintos aunque vinculen lo mismo. Es la única prueba que mira adentro de la clausura, y por eso la única que nota el cambio de representación. Reescrita:

```
(define clausura-y
  (val->proc (value-of (proc-exp 'y (diff-exp (var-exp 'y) (var-exp 'z)))
                       (extend-env 'z (intval 3) ρx))))

(check-equal? (apply-env (closure-ρ clausura-y) 'z)
```

```
              (intval 3)
              "[proc] la clausura guarda las variables libres del cuerpo")

(check-false (has-binding? (closure-ρ clausura-y) 'x)
             "[proc] y no guarda las que el cuerpo no usa")
```

Sin `has-binding?`, `let f = proc (x) y in 5` sería error al crear el procedimiento, porque `apply-env` no encuentra `y`. En `proc-02` el error llega solo si se llama `f`. Recortar el entorno no puede adelantarlo: sería un programa con otro resultado.

```
(check-equal? (run "let f = proc (x) y in 5")
              (intval 5)
              "una variable libre sin vincular no es error al crear el procedimiento")
```